

7.15 Stack 08 B — preflight and static-analysis audit

Identity and scope

Reviewer: Codex, sole audit agent, 2026-09-27 UTC. Unit B is the second and final unit in the one owner-authorized binary breakdown. Base is unit A at 1172cd309a280845c8d14c372ee45ec7d343e207 on release/715-stack-08a-evidence; code snapshot c519c0be98a3529b09800c6da4bc78b80f92368e, branch release/715-stack-08b-preflight. The containing report commit and live CI/review IDs are in the PR receipt. Python #120 remains d9a02d981ee43fe3e80b28ba02466670e6e8409d, with clean review 5329239805. All other dependency pins and production code are unchanged. A's broader wire/report audit and historical 23-path mapping are inherited evidence; the complete adjacent B inventory below is the review boundary.

Owner budget: initial 3/3, one binary split used, A 3/3 and checkpoint complete under the SOP's actionable-finding rule. A final review 5329724450 says Findings: None with no inline findings or unresolved threads; its generic overview identified no concrete issue. B has used 1/3 requests. No further split is authorized. Python #120 remains clean. Aggregate #872 carries the combined tree plus its receipt. Non-PR report runs require the independently configured accepted repository SHA or fail closed; no settings were changed.

Findings and repairs

Finding	Disposition
4114517060: command extraction allegedly mangles continuations and removes the dollar sign from nproc	The alleged corruption did not reproduce: the recorded prior container process used cppcheck once with -j 16 and completed cleanly. The text-extraction design was nevertheless fragile. It is removed entirely. CI and local Docker now invoke the same scripts/cppcheck.sh, which builds argv directly with shell quoting and no eval or extracted command text.
4114517076: cached unpinned analyzer may differ from CI	Both paths now install Ubuntu package 2.13.0-2ubuntu3. CI pins the Ubuntu 24.04 runner; local Docker uses Ubuntu 24.04. The shared invocation checks both dpkg's installed package version and the actual executable's Cppcheck 2.13.0 output on every invocation, including a reused image. A stale image or shadowed executable fails before analysis.
4114517108: report suite executes twice	Preflight executes the suite once, retains full diagnostics and uses that invocation's exit status. -B avoids writing Python cache files into the source tree.

A owns the other three findings in firmware review 5329364719: platform-selected waiver authority, empty ledger declarations and exact product skip reasons. They are outside this adjacent diff and are mapped in A's report. Earlier B08-004 and its follow-up are preserved in A's evidence rather than silently discarded.

Static-analysis execution and failure behavior

The shared script preserves CI's analysis flags, includes, defines, output template, source directories and nonzero-on-finding policy. It requires one literal output pathname. Parallelism is obtained inside the execution environment with getconf, not parsed from YAML. The local container mounts source read-only and gives only the analysis cache a writable temporary mount; diagnostics go to the container's temporary directory. Container or analyzer failure is propagated directly. A missing package/image, stale package, wrong executable, missing Docker or failed test prevents preflight success.

Package version is pinned and checked, not inferred from a cached image tag. Distribution image updates can still alter unrelated OS dependencies; the claim is identical analyzer package/version and argv, not byte-identical operating systems across host architectures. The CI runner and local container may have different CPU architectures. Exact-head CI remains required acceptance evidence in addition to local preflight.

Verification

Five focused tests use a controlled executable to capture actual argv and exit status: source/analysis flags and a literal output path containing spaces and shell syntax; rejection of stale package; rejection of shadowed executable; propagation of analyzer exit 7; rejection of missing/extra output arguments. All five pass. Five additional preflight boundary tests pass; with the inherited 18 report-gate tests, 28 tests pass in total. Shell syntax and workflow lint pass. The full preflight is required after this report freeze and before push; its result is recorded in the live receipt.

Firmware runtime, Python production/library/test code, native suites and the Python pin are unchanged by B. The already qualified Stack 08 wire/ABI behavior is inherited: 65 local wire/runtime tests, 39 compiler/catalog/protocol/report tests, 657 native tests, 11 inspected EX42 frames and all 39 predecessor runtime methods preserved. Full CI on aggregate 68f0b1824 passed 1,497 tests with 89 expected skips and zero failure/error. B must still produce its own full/BTC ARM/emulator, native, integration and report evidence, with exact firmware/Python SHAs, PRs, run URL and PDF digest verified before acceptance.

Review and remaining boundaries

The three inherited B findings were repaired before the first B request; the additional first-round findings are repaired below before the second request. The earlier review's exact command-corruption allegation is technically declined using observed argv, while the fragile extraction mechanism is removed. The new checkpoint must cover these actual shared-script/preflight/CI changes against A; a report-only review is insufficient. Current-head clean review, zero unresolved threads and exact-head CI evidence are required. Physical-device validation, assembled release acceptance, product merges and publication remain separate gates.

B round 1 findings and verification

Review 5331723008 found two inline issues plus concrete body-only suggestions. The basename search could accept an unrelated file or a comment; preflight now configures the actual test CMake tree with full features and compares tracked test paths against compile_commands.json. Linking is omitted and firmware is not compiled by this source-membership check; real set/list/iff/add_subdirectory semantics remain. Repository-root source paths are exposed through temporary directory aliases. Missing CMake or configuration errors fail closed. This checks membership in the configured full-feature test graph, not runtime test success; native CI remains required.

GNU timeout is replaced by Python subprocess.run(timeout=20), which works on macOS and reports missing, failed or timed-out Docker probes. Whitespace checks use the merge base with the upstream plus the working tree, covering committed changes about to be pushed; PREFLIGHT_BASE explicitly selects the adjacent review base or is required for a first push without an upstream. CI creates the analyzer report before invocation so early package/executable failure still leaves an uploadable report. The analyzer cache is ignored locally. The argv regression asserts every flag, include, define, template, output path and source directory, allowing only the host CPU count to vary.

Five new boundary tests exercise a same-basename collision, comment and unused-list rejection, a repository-root CMake source, committed/worktree whitespace, absent upstream, and Docker timeout/failure behavior. All 28 combined preflight/analyzer/report tests pass, and the structural check passes on the real repository. Initial harness validation caught a CMAKE_SOURCE_DIR relocation problem; temporary directory aliases fixed it and a real-CMake fixture now covers that case. No production firmware, runtime suite, Python pin or A code changed. Final preflight must run after this source/PDF freeze, followed by new exact-head CI/report verification and B request 2/3. Earlier successful runs describe earlier B/aggregate heads.

Complete adjacent inventory

Added	Deleted	Path
6	26	.github/workflows/ci.yml
3	0	.gitignore
58	0	docs/release/audit-units/715-08B-audit-20260927.md
-	-	docs/release/audit-units/715-08B-audit-20260927.pdf
1	0	scripts/cppcheck-version
32	0	scripts/cppcheck.sh
103	0	scripts/preflight.sh
79	0	scripts/preflight_checks.py
76	0	scripts/test_preflight_checks.py
91	0	scripts/test_preflight_cppcheck.py